

RNN-Transformer 实现文本分类实验指导

实验概览

实验目的

- 理解循环神经网络（RNN）和 Transformer 的基本原理和结构。
- 掌握使用 RNN 和 Transformer 模型对文本数据进行分类的方法。
- 学习如何评估 RNN 和 Transformer 模型的分类性能，并调整网络参数以优化性能。
- 掌握中英文文本数据的预处理方法和特征提取技术。

实验要求

- 使用 PyTorch 和 torchtext 库完成数据加载与预处理。
- 分别实现基于 RNN 和 Transformer 的文本分类模型。
- 对模型进行训练、验证与测试，记录并分析实验结果。
- 可选部分：使用中文新闻数据进行分类，比较中英文文本处理的异同。

提交内容

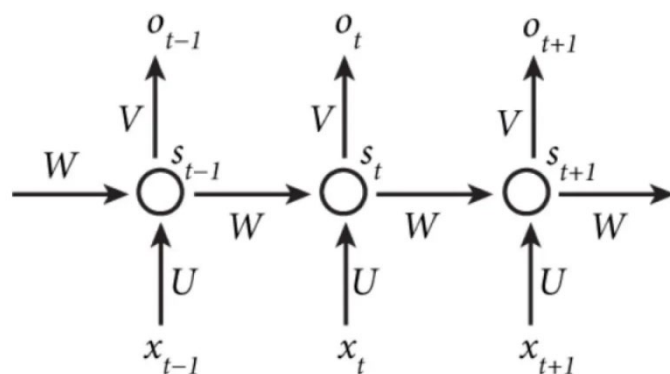
实验报告（包含实验目的、步骤、结果分析等），采用 PDF 格式。报告应结构清晰、内容详实、语言流畅。

循环神经网络（RNN）

循环神经网络（Recurrent Neural Network，**RNN**）是一种专门用于处理序列数据的神经网络结构。**RNN** 通过其循环连接能够捕捉序列中的**时间依赖关系**，使其非常适合处理文本、语音等序列数据。

网络结构

RNN 包含**循环连接**，使得网络能够维护一个**隐藏状态**，该状态在每个时间步更新并传递信息。常用的 **RNN** 变体包括 **LSTM**（长短期记忆网络）和 **GRU**（门控循环单元），它们通过**门控机制**解决了传统 **RNN** 的梯度消失问题。



梯度爆炸



现象：训练过程中梯度值急剧增大，导致模型参数更新过大而无法收敛。

解决方法：采用**梯度截断**的方式

通过设置梯度的阈值，将超过阈值的梯度缩放到一定范围内，从而避免梯度爆炸的发生。

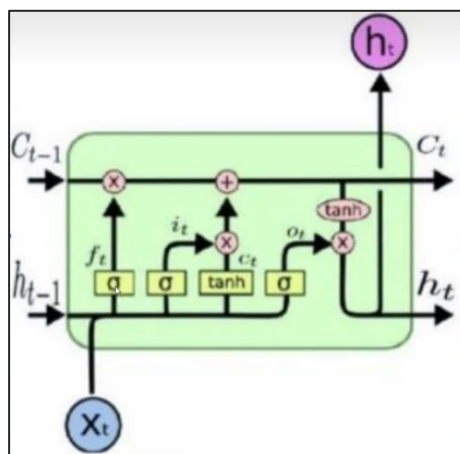
```
for idx, (label, text, offsets) in enumerate(data_loader):
    optimizer.zero_grad()
    predicted_label = model(text)
    loss = criterion(predicted_label, label)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.1)
    optimizer.step()
    total_acc += (predicted_label.argmax(1) == label).sum().item()
    total_count += label.size(0)
```

梯度消失



现象：梯度值随着时间步传播而指数级减小，导致远距离依赖难以学习。

解决方法：使用 LSTM



LSTM 缓解梯度消失的原理：

LSTM（长短期记忆网络）通过**细胞状态（Cell State）**的线性传播机制和**门控机制（Gating Mechanism）**的自适应调节，从根本上改变了梯度在反向传播中的流动方式，从而有效缓解了 RNN（循环神经网络）中的梯度消失问题。

Transformer

在深度学习领域，尤其是自然语言处理任务中，如何有效地对序列数据进行建模一直是一个核心问题。传统的循环神经网络及其变体（如 LSTM）在处理序列时存在固有的局限性。Transformer 模型的提出，完全摒弃了循环结

构，转而基于**自注意力机制**构建，从根本上解决了这些问题，成为了当前大语言模型的基石。

自注意力机制

自注意力机制是 Transformer 模型理解序列信息的核心组件。它的设计灵感来源于一个直观的想法：当我们在处理一个句子中的某个词时，为了更好地理解它，我们需要去看**句子中的其他所有词**，并根据相关性赋予不同的关注度。

1. **核心功能**：计算序列中每个位置与**其他所有位置的相关性**。
2. **核心目的**：捕捉长距离依赖关系。
3. **核心价值**：解决了 RNN 在处理长序列时的局限性。

位置编码

由于自注意力机制本身不具备感知词序的能力（它对序列的排列是不敏感的），Transformer 需要一种方法来明确地告诉模型每个词的位置。

- **为什么需要？** 如果没有位置信息，模型会将“狗追猫”和“猫追狗”视为相同的输入。
- **如何实现？** Transformer 使用**位置编码**——一组**基于正弦和余弦函数**生成的、具有特定模式的向量。这些向量包含了位置信息，会与词的嵌入向量**相加**，作为模型的最终输入。这样，模型就能同时利用词的语义和其在序列中的位置。

Transformer 的优势

- 并行计算能力强，训练效率高
- 长距离依赖捕捉能力优异
- 可扩展性好，适合大规模预训练

RNN 实现英文文本分类

1. 数据准备

(1) 下载数据集

安装 `torchtext` 库，并下载 `AG_NEWS` 数据集。下载完成后观察数据集形式，统计数据集的基本信息（如平均长度、分类数等）。

注意！ 本次实验建议新建一个 `conda` 虚拟环境，并安装以下版本，防止各个包发生版本冲突：

```
torchdata==0.5.1
torch==1.13.1
torchtext==0.14.1
```

```
import torch
from torchtext.datasets import AG_NEWS
train_iter = iter(AG_NEWS(split='train'))

from torchtext.data.utils import get_tokenizer
from torchtext.vocab import build_vocab_from_iterator
```

（2）数据预处理

对数据进行预处理以提高网络的训练效果。

2.网络构建

使用 LSTM 实现文本分类。首先将文本通过 `embedding` 转换成连续的矩阵，然后使用双向循环神经网络提取特征，再接全连接层，实现分类预测。

3. 模型训练与评估

（1）训练过程执行

- 设置合适的训练轮数（`epochs`）和批量大小（`batch size`）。
- 将训练集数据输入网络，进行前向传播和反向传播。
- 根据损失函数的梯度更新网络的权重。
- 监控训练过程中的损失和准确率，以评估网络的训练效果。

（2）测试集评估模型

在测试集上评估模型性能，并保存相应的模型参数。

```
ex_text_str = "MEMPHIS, Tenn. Four days ago, Jon Rahm was \
enduring the season's worst weather conditions on Sunday at The \
Open on his way to a closing 75 at Royal Portrush, which \
considering the wind and the rain was a respectable showing. \
Thursday's first round at the WGC-FedEx St. Jude Invitational \
was another story. With temperatures in the mid-80s and hardly any \
wind, the Spaniard was 13 strokes better in a flawless round. \
Thanks to his best putting performance on the PGA Tour, Rahm \
finished with an 8-under 62 for a three-stroke lead, which \
was even more impressive considering he'd never played the \
front nine at TPC Southwind."

# model = model.to("cpu")

print("This is a %s news" % ag_news_label[predict(ex_text_str, text_pipeline)])
```

(3) 结果分析与优化

- 尝试调整网络参数，如隐藏层数量、神经元数量、学习率等，以优化分类性能。
- 记录并比较不同参数设置下的实验结果

Transformer 实现英文文本分类

1. 数据准备

(1) 下载数据集

安装 `torchtext` 库，并下载 `AG_NEWS` 数据集。下载完成后观察数据集形式，统计数据集的基本信息（如平均长度、分类数等）。如有 `gpu` 可安装 `cuda` 版本。

注意！ 本次实验建议新建一个 `conda` 虚拟环境，并安装以下版本：

```
torchdata==0.5.1
torch==1.13.1
torchtext==0.14.1
```

(2) 数据预处理

对数据进行预处理，以提高网络的训练效果。

2.网络构建

(1) 定义网络结构

首先将文本通过 embedding 转换成连续的矩阵，然后使用 Transformer 提取特征，再接全连接层，实现分类预测。

4. 模型训练与评估

(1) 训练过程执行

- 设置合适的训练轮数（epochs）和批量大小（batch size）。
- 将训练集数据输入网络，进行前向传播和反向传播。
- 根据损失函数的梯度更新网络的权重。
- 监控训练过程中的损失和准确率，以评估网络的训练效果。

(2) 测试集评估模型

在测试集上评估模型性能，并保存相应的模型参数。

(3) 结果分析与优化

- 尝试调整网络参数，如隐藏层数量、神经元数量、学习率等，以优化分类性能。
- 记录并比较不同参数设置下的实验结果
- 总结全连接神经网络在鸢尾花数据集上的分类性能。

【可选】

RNN、Transformer 实现中文文本分类

使用中文新闻数据，首先使用分词工具（如 jieba 等）先对中文进行分词，然后使用上述方法，测试在中文数据上的性能。